

Smart Attendance Verification System

Master Technical Blueprint

RFID Check-In + BLE Dwell-Time Tracking (40+ Minute Verification Logic)

Team: 1 Hardware Engineer + 1 Software Engineer

Architecture · Backend · Database · MQTT · Algorithms · Security · Testing · Roadmap

Companion to reference document: Smart Attendance Verification System — Project Cost

Generated: September 2026

Smart Attendance Verification System

Master Technical Blueprint — RFID Check-In + BLE Dwell-Time Tracking (40+ Min Verification)

Project type: Embedded IoT & Automation · Target: Smart Classrooms & Lecture Halls · Team: 1 hardware + 1 software

Companion to reference document: "Smart Attendance Verification System — Project Cost" (₹1,350-1,950 / node)

Legend used throughout: **[PDF]** = required by the reference PDF · **[RESEARCH]** = external sources/documentation · **[REC]** = recommended implementation · **[ASSUMPTION]** = gap resolved by assumption

1. Executive Summary

The PDF describes a fundamentally sound two-layer concept: **RFID tap (intent) + BLE dwell-time (presence)**. The RFID layer is completely buildable and reliable. The BLE layer, as specified — "the student's BLE beacon/smartphone signal" — contains the project's single largest technical risk: **smartphones cannot reliably act as continuously-advertising BLE beacons in 2026**, especially iPhones. This does not kill the project; it means the "student device" side needs one pragmatic modification (a cheap beacon keyfob or an Android-only foreground-service app), while everything else — ESP32, RC522, MQTT, backend, dashboards — stays exactly as planned.

Everything below is designed for one software developer + one hardware developer, real classroom deployment, hackathon demo-ability, and zero unnecessary complexity.

2. Understanding of the PDF

[PDF] The system is a single-node-per-classroom device: ESP32-WROOM-32 dev board (Wi-Fi + BLE 4.2/5.0), RC522 RFID reader (13.56 MHz, SPI) at the entry door, 0.96" I2C OLED + buzzer + LEDs for feedback, wall-mounted at the door, mains-powered (5V/2A).

[PDF] Core logic (two layers):

- **Layer 1:** RFID tap → session created → state CHECK_IN
- **Layer 2:** ESP32 scans BLE; if the student's signal RSSI ≥ -75 dBm, accumulate dwell time (+10 s per detection interval); short breaks < 5 min pause accumulation; at 2,400 s cumulative → VERIFIED → PRESENT logged to cloud
- FSM: IDLE → CHECK_IN → MONITORING → VERIFIED
- Session length: 50-60 min; target dwell: 40 min
- Connectivity: Wi-Fi → MQTT (Mosquitto/HiveMQ) or HTTP REST; local EEPROM/Flash backup
- Web stack mentioned: Node-RED, React, PostgreSQL
- Cost: ₹1,350-1,950 (~\$16-23) per node; RFID cards ₹15-20 each; optional BLE beacons + ₹250-350 each

[PDF] "Wall bleed mitigation": calibrate RSSI threshold between -70 and -78 dBm to reject hallway/adjacent-classroom signals.

3. Requirements Extracted from the PDF

A. System requirements — one ESP32 node per classroom, wall-mounted at entry door; detect RFID tap (entry intent), then verify sustained presence via BLE for ≥ 40 of 50-60 min; push results to cloud database; survive Wi-Fi outages with local backup.

B. Functional requirements — read RFID UID and identify student; create attendance session on tap; continuously scan BLE and match student's device; accumulate dwell time only when RSSI $\geq$ threshold; pause (not reset) on short absences (< 5 min); mark PRESENT at 2,400 s, otherwise ABSENT; display status on OLED; beep/LED feedback on tap; sync to server via MQTT or HTTP.

C. Non-functional requirements — real-time-ish feedback at the door (< 2 s tap-to-feedback); offline resilience (local Flash queue + replay); low cost per node; scalable to many classrooms; cloud integration with real-time institutional analytics.

D. Hardware requirements — ESP32-WROOM-32, RC522, OLED, buzzer, LEDs, 5V/2A supply, enclosure.

E. Software requirements — FSM on ESP32, MQTT/HTTP client, cloud database, web dashboard, calibration tooling.

F. Communication requirements — Wi-Fi 802.11 b/g/n, MQTT (or HTTP REST); QoS behavior unstated **[ASSUMPTION]**; JSON payloads implied.

G. Data flow: RFID card → RC522 → ESP32 → MQTT/HTTP → Backend → PostgreSQL → Web dashboard → Admin/Teacher/Student. Same path for BLE device → ESP32 BLE scanner.

4. Current Architecture (as proposed)

[Student RFID Card]

[Student phone/beacon]

```

| tap | BLE adv
[RC522] --SPI--> [ESP32] <--BLE scan---+
| Wi-Fi + MQTT/HTTP
v
[MQTT Broker / HTTP API]
v
[Cloud Database (PostgreSQL)]
v
[Node-RED / React Dashboard]

```

The architecture is directionally fine. The problems are inside the boxes, not between them (details in §5).

5. Problems / Risks in the Proposed Architecture

#	Problem	Severity	Tag
1	"Student's smartphone as BLE beacon" is unreliable on iOS and fragile on Android (see §11). A student with an iPhone essentially cannot participate as specified.	CRITICAL	[RESEARCH]
2	MAC-address-based identification breaks — phones randomize BLE MACs (recommended rotation < 15 min), so "unique signal per student" cannot rely on MAC.	CRITICAL	[RESEARCH]
3	RFID UID is trivially cloneable — MIFARE Classic UID can be copied with ~\$30 tools or a Flipper Zero; RC522 reads UID only.	HIGH	[RESEARCH]
4	RSSI ≥ -75 dBm as a presence gate is a single noisy binary decision — RSSI fluctuates ± 10 –15 dB with body orientation, pockets, multipath.	HIGH	[RESEARCH]
5	Wi-Fi/BLE coexistence on ESP32 — single 2.4 GHz radio, time-division multiplexed; BLE scans get interrupted by Wi-Fi traffic; misconfigured scan windows starve Wi-Fi.	HIGH	[RESEARCH]
6	"+10 s per interval" accumulation is underspecified — scan interval, missed-scan handling, clock drift, duplicate events undefined.	MEDIUM	[PDF] ambiguity
7	"EEPROM/Flash local backup" — the classic ESP32 EEPROM library is deprecated emulation; NVS or LittleFS is the correct choice.	LOW	[RESEARCH]
8	Dwell state lives "in ESP32 RAM" — power loss mid-session loses all dwell time.	MEDIUM	[PDF]
9	No device identity/authentication, no TLS, no ACL — anyone on the network can publish fake attendance.	HIGH	[PDF] omission
10	Node-RED listed as a dashboard option — it would become a second, overlapping system; pick one stack.	LOW	[REC]

What must stay exactly as specified [REC]: the two-layer concept, the FSM idea, the 40-min / 2,400 s dwell target, the -70 ... -78 dBm calibration range (as a starting band, refined by §12/§16), the hardware BOM (it is fine), MQTT as transport, PostgreSQL, React.

6. Existing Technologies & Research

1-5. RFID + BLE presence systems. RFID check-in is a solved, reliable problem — RC522 + MFRC522 library + 4-byte UID is the standard path. Presence-via-BLE-RSSI is an active research area with an honest consensus: **RSSI alone gives room-level, not seat-level, presence, and is noisy at room scale.** Systems like ESPresense (ESP32 room-assistant) prove ESP32 RSSI tracking works for "which room is this tag in" — the exact use case — but they use **dedicated tags with fixed MACs**, not phones.

6. ESP32 + RC522 integration. Mature: SPI at 3.3 V, miguelbalboa/rfid library, UID read in ~ 50 ms. Known caveats: keep SPI

wires short, 2–5 cm read range.

7-9. MQTT architectures. The de-facto IoT protocol. Mature, lightweight, pub/sub with topic ACLs, TLS, LWT. The OASIS spec is stable.

10. IoT device auth. Standard pattern: per-device username/password (or client cert) into broker ACLs, restricting publish to devices/{id}/# and subscribe to devices/{id}/cmd|ack only.

11-14. Attendance systems / anti-proxy. Academic literature converges on the same finding as the PDF: single-factor RFID fails to proxy tapping; adding a second, independent presence factor (dwell time, Wi-Fi probe frames, or camera verification) materially raises cheating cost.

15-16. Smartphone BLE advertising & privacy. This is where the PDF's assumption collides with 2026 reality:

- **iOS:** a backgrounded app **cannot advertise** a custom GATT service UUID at all; only the restricted "overflow area" mechanism exists, and receiving it requires the receiving device's screen to be on. Apple's own docs: background advertising drops the local name and moves service UUIDs to the overflow area, and advertising frequency may decrease. Independent analysis confirms: "In the foreground, it can emit an iBeacon advertisement... In the background, it can do none of these things."
- **Android:** persistent advertising requires a **foreground service with a permanent notification**; even then, OEMs (especially Samsung "Adaptive Battery"/"Sleeping apps") kill background processes within minutes, and advertisements can silently stop even after onStartSuccess.
- **MAC randomization:** BLE LE Privacy rotates addresses (spec recommends < 15 min) specifically to prevent tracking — so a fixed MAC → student mapping is unreliable and a privacy anti-pattern. iOS randomizes on every wake.

17. RSSI indoor localization. Well-studied: RSSI is affected by walls, bodies (human tissue attenuates 2.4 GHz ~3–10 dB), orientation, multipath, and interference. Usable for room-level occupancy with smoothing and hysteresis; never a precise distance meter. BLE advert payloads are capped at 31 bytes (legacy), so the identity token must be tiny or use a registered service UUID + short manufacturer data.

18. Edge vs. cloud verification. The PDF accumulates dwell on the ESP32 (edge). **[REC]** Move the **authoritative** accumulation to the backend (cloud); keep the ESP32 as a sensor that uploads *observations*. Edge-only accumulation loses state on reboot, cannot be audited, and cannot be corrected by a teacher.

ESP32 coexistence [RESEARCH]: the ESP32 has **one shared 2.4 GHz radio** for Wi-Fi and BLE; Espressif implements time-division multiplexing, and "BLE SCAN may be interrupted by Wi-Fi." Practical rule from the field: BLE scan window ≤ 50% of scan interval when Wi-Fi is active. Use **NimBLE** (not Bluedroid): dramatically lower RAM, recommended across the community.

MQTT QoS [RESEARCH]: QoS 1 guarantees delivery but **not** dedup — duplicate messages are inevitable at protocol level; QoS 2 is exactly-once but 2× packets and heavier broker state. The standard industrial pattern is **QoS 1 + application-level idempotency keys**, never QoS 2.

RFID cloning [RESEARCH]: "UID-based systems are easy to clone." A Flipper Zero or \$30 UID-writable card clones an RC522-readable card in seconds. Mitigation is cheap: sign a card-block secret.

7. Technology Comparison

Layer	Options	Verdict for 2-person team
Frontend	React / Next.js / Vue / plain JS	React + Vite + TS. Next.js adds SSR you do not need; plain JS loses type safety for little gain.
Backend	Express / NestJS / FastAPI / Django	Node.js + Express + TypeScript. One language end-to-end, huge BLE/MQTT ecosystem, fastest iteration. FastAPI is equally fine if you prefer Python — pick one and do not reconsider.
Database	PostgreSQL / MySQL / MongoDB / SQLite	PostgreSQL. JSONB for event payloads, mature, free, matches the PDF.
Realtime	WS / Socket.IO / SSE / polling	WebSocket (Socket.IO). Dashboards are bidirectional-light; SSE cannot push from client; polling is a fallback only.
IoT transport	MQTT / HTTP / WS	MQTT for device→cloud (events/heartbeat); REST for human→cloud (dashboards). Hybrid — do not force one protocol to do both.
Broker	Mosquitto / EMQX / HiveMQ	Eclipse Mosquitto. Mature, tiny, ACL + TLS built in, runs in Docker, zero cost.
Auth	JWT / sessions / OAuth	JWT access (15 min) + refresh token in httpOnly cookie + RBAC roles (admin/teacher/student).
Deployment	Docker / VPS / Render / Railway / AWS	One small VPS + Docker Compose (Hetzner CX22 / DO \$6 tier) with Mosquitto, API, Postgres, Caddy. Railway/Render as zero-DevOps fallback.
Language	TS vs JS	TypeScript everywhere — the payloads and DB schemas in this project are exactly where types pay off.
ORM	Prisma / Drizzle / raw SQL	Prisma — schema as source of truth, migrations included.
Validation	Zod (shared)	Validate at the API edge with Zod schemas

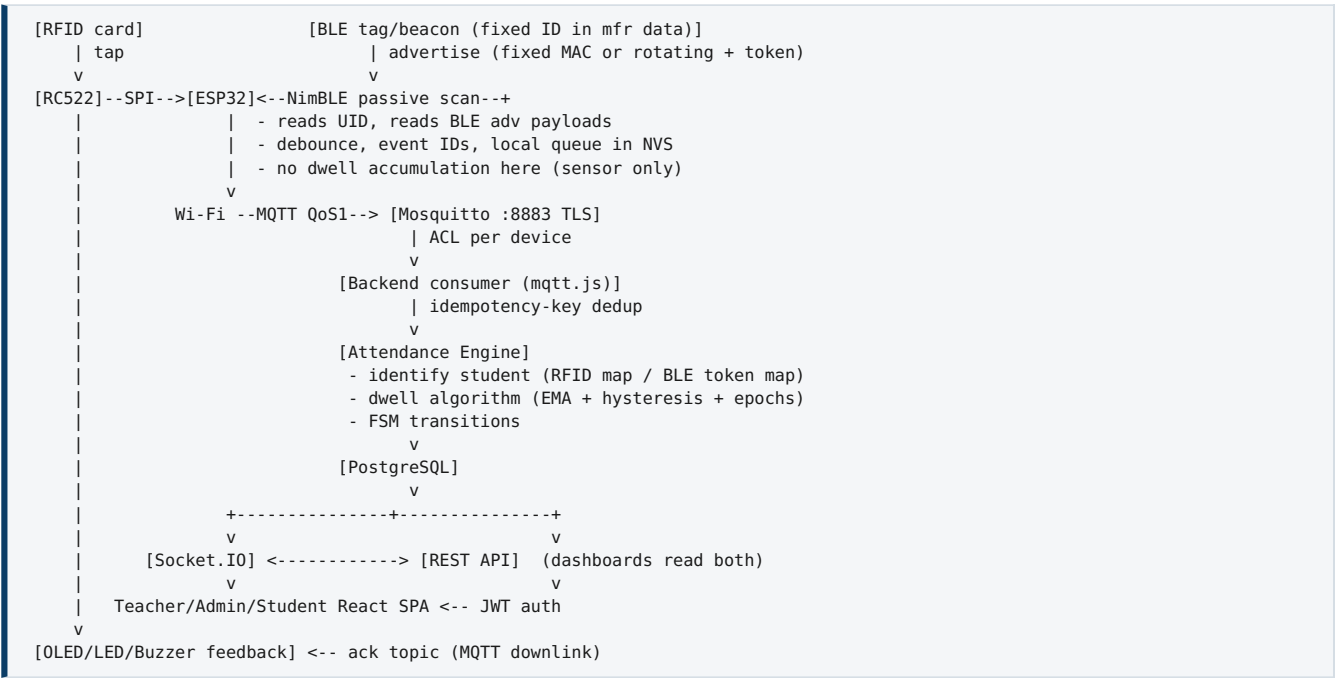
	shared frontend/backend.
--	--------------------------

"Why do we need this? / Why better than simpler?" for the only debatable calls: MQTT (not pure REST) because the ESP32 gets store-and-forward, QoS acks, LWT offline detection, and 1:N fan-out for free — HTTP would force you to build retry queues and liveness yourself. WebSocket (not SSE) because teacher commands (e.g., "end session early") flow client→server later. Docker (not bare metal) because "works on my laptop" must survive the demo.

8. Recommended Tech Stack

Frontend:	React 18 + Vite + TypeScript + TailwindCSS + TanStack Query + Socket.IO-client + Recharts
Backend:	Node.js 20 + Express + TypeScript + Zod + Prisma + Socket.IO + mqtt.js + jsonwebtoken + bcrypt + pino (logging) + zod-validated env via envalid
Database:	PostgreSQL 16
IoT protocol:	MQTT (device uplink) + REST (web/API)
MQTT broker:	Eclipse Mosquitto 2 (Docker), TLS on 8883, per-device ACLs
Auth:	JWT access (15 min) + refresh cookie, RBAC: ADMIN TEACHER STUDENT
Realtime:	Socket.IO (WebSocket)
Deployment:	Single VPS + Docker Compose + Caddy (auto-TLS); GitHub Actions -> SSH deploy
Monitoring:	pino logs + /healthz; Prometheus optional (NICE TO HAVE); MQTT Explorer (dev tool)
Testing:	Vitest (unit), Supertest (API), Playwright (E2E, optional), simulator (device tests)
Firmware:	Arduino core for ESP32 + NimBLE-Arduino + PubSubClient (or ArduinoMqttClient) + Preferences (NVS)

9. Final System Architecture



Key deliberate choice: **the ESP32 is a dumb, robust sensor; the backend is the brain.** This fixes problems #6, #8, #9 from \$5, and enables teacher correction, auditing, and hardware-free development via a simulator.

10. Hardware <-> Software Responsibility Split

Member 1 (Hardware)	Member 2 (Software)
Soldering, wiring, enclosure, PSU	Backend, DB, APIs, auth
ESP32 firmware: RC522 read, NimBLE scan, debounce	MQTT broker + ACLs + TLS
Local queue (NVS) + reconnect/replay	Attendance engine + FSM (server-side)
OLED/LED/buzzer feedback on ack	Simulator (fake device over MQTT)
Heartbeat publishing	Web dashboards (admin/teacher/student/live)
RSSI walk-test data collection in real classroom	RSSI calibration analysis -> threshold config
Power/reset behavior, brownouts	Deployment, Docker, CI, testing
Device provisioning (flash deviceId/credentials)	Device registry in DB matching those credentials

Contract between members: **a frozen MQTT payload schema + topic spec** (§13). Once agreed, both work in parallel.

11. Backend Architecture

```
backend/
+-- src/
|   +-- config/      # env.ts (zod-validated), prisma client
|   +-- mqtt/        # broker client, consumer, publisher (acks)
|   +-- modules/
|   |   +-- auth/     # routes, service, jwt, middleware
|   |   +-- users/    # admin/student/teacher CRUD
|   |   +-- devices/  # ESP32 registry, heartbeat monitor
|   |   +-- classes/  # courses, classrooms, sessions
|   |   +-- attendance/ # engine, FSM, dwell calculator
|   |   +-- events/   # raw rfid_events / ble_observations (query + debug)
|   +-- realtime/    # Socket.IO server, room-per-session fanout
|   +-- jobs/         # session auto-start/end sweeper, heartbeat staleness, dwell tick
|   +-- middleware/   # authN, authZ(RBAC), error handler, request logger
|   +-- app.ts        # express app, route mounting
+-- prisma/schema.prisma
+-- tests/
+-- Dockerfile
```

- **Controllers:** thin — parse/validate (Zod), call service, return.
- **Services:** all business logic; the Attendance Engine is pure functions over events -> state transitions (unit-testable without MQTT or DB).
- **MQTT consumer:** subscribes school/att/devices/+/evt|hb; parses; writes events tables; hands typed events to the engine; publishes acks.
- **Jobs (node-cron):** (1) sweep class_sessions past end-time -> finalize; (2) mark devices stale if last heartbeat > 90 s; (3) dwell tick every 15 s to recompute epoch presence.
- **Logging:** pino JSON, request-id propagated, device logs tagged deviceId.
- **Errors:** central error middleware -> consistent { error: { code, message } }; MQTT parse failures -> dead-letter table, never crash the consumer.
- **Validation:** every REST body validated by Zod; MQTT payloads validated with the same schemas (shared).

12. Database Architecture

Pragmatic schema — 11 tables. Everything else (audit, history) hangs off these.

```
users --+--< students
      +--< teachers

classrooms --< devices (1:N; each ESP32 node belongs to one room)
rfid_cards --1:1-- students      (uid unique; card-<->student mapping)
ble_identities --1:1-- students  (token/UUID + optional fixed MAC)

courses --< class_sessions --< attendance_sessions --< attendance_results

class_sessions --< rfid_events
class_sessions --< ble_observations
devices --< device_heartbeats
audit_logs (polymorphic: actor, action, entity, entityId, before/after JSONB)
```

Core columns (abbreviated DDL — the Prisma schema is the real artifact):

```
users(id uuid pk, email citext unique, password_hash, role enum('ADMIN','TEACHER','STUDENT'),
      created_at, updated_at)

students(user_id pk fk->users, roll_no varchar unique, department, semester)

rfid_cards(uid varchar(16) pk, student_id fk unique, active bool, issued_at)

ble_identities(id uuid pk, student_id fk unique, token_hash varchar unique, -- token = rotating secret in adv payload
               fixed_mac macaddr null, -- for beacon keyfobs
               active bool)

classrooms(id uuid pk, name varchar unique, building, room_no varchar unique)

devices(id varchar(32) pk, -- e.g. "ESP32-CR01-A3F2" (printed on case)
        classroom_id fk, mqtt_username varchar unique, firmware_ver varchar,
        status enum('ONLINE','DEGRADED','OFFLINE'), last_seen timestampz,
        created_at)

courses(id uuid pk, code varchar unique, name, teacher_id fk->teachers)
```

```

class_sessions(id uuid pk, course_id fk, classroom_id fk, teacher_id fk,
  starts_at, ends_at, status enum('SCHEDULED','LIVE','CLOSED'),
  dwell_required_secs int default 2400, rssi_enter int default -70,
  rssi_exit int default -78, unique(course_id, starts_at))

attendance_sessions(id uuid pk, class_session_id fk, student_id fk,
  state enum('CHECKED_IN','MONITORING','TEMP_ABSENT','VERIFIED','UNVERIFIED','CLOSED'),
  checked_in_at timestamptz, dwell_secs int default 0,
  last_seen_at timestamptz, verified_at timestamptz null,
  unique(class_session_id, student_id))

rfid_events(id bigserial pk, class_session_id fk, device_id fk, student_id fk null,
  uid varchar(16), event_id varchar(40) unique,      -- idempotency key from ESP32
  ts_device timestamptz, ts_server timestamptz, result varchar(32))

ble_observations(id bigserial pk, class_session_id fk, device_id fk,
  student_id fk null, rssi smallint, identity_token varchar null,
  epoch_no int, ts_device timestamptz, ts_server timestamptz)
  index (class_session_id, student_id, epoch_no)

device_heartbeats(id bigserial pk, device_id fk, uptime_secs int, free_heap int,
  rssi_wifi smallint, ip inet, firmware_ver varchar, ts timestamptz)

audit_logs(id bigserial pk, actor_id uuid, actor_role varchar, action varchar,
  entity varchar, entity_id varchar, before jsonb, after jsonb, ts timestamptz)

```

Why these tables: rfid_events and ble_observations are immutable raw telemetry (debugging + algorithm tuning gold); attendance_sessions is the live FSM state; attendance_results can simply be a VIEW over final attendance_sessions — do not duplicate state. Idempotency lives on event_id unique constraints — duplicate MQTT deliveries become unique-violation no-ops.

13. MQTT Architecture

Topic hierarchy (single-tenant; school/att prefix):

```

UPLINK (device -> broker, ESP32 publishes):
  school/att/devices/{deviceId}/evt      QoS 1  RFID taps + BLE batch observations
  school/att/devices/{deviceId}/hb      QoS 0  heartbeat every 30 s

DOWNLINK (backend -> device, ESP32 subscribes):
  school/att/devices/{deviceId}/ack      QoS 1  per-event ack (LED/OLED/buzzer result)
  school/att/devices/{deviceId}/cmd      QoS 1  config: thresholds, session start/stop, time sync

STATE (broker-managed):
  school/att/devices/{deviceId}/status  QoS 1, RETAINED  "online"/"offline" via LWT

```

Design rules [REC]:

- **QoS 1 everywhere except heartbeat (QoS 0)** — matches "QoS 1 + idempotency key" best practice; QoS 2 never (2x overhead, broker state, no benefit with idempotent payloads).
- **Every uplink event carries eventId** (ESP32-generated UUID or monotonic counter + bootId). Backend dedups on event_id unique -> replay after outage is safe.
- **ESP32 connects with cleanSession=false** so offline cmd/ack messages queue.
- **LWT** = retained status:"offline" on status topic; connect publishes retained status:"online". Dashboard reads retained messages -> instant fleet view.
- **Batch BLE observations** every 15 s into one evt message (type ble_batch) instead of 40 messages/sec — this directly reduces Wi-Fi/BLE radio contention (§5 #5).
- **Authentication:** per-device mqtt_username = deviceId, password from device provisioning. **ACL:** device may publish only to school/att/devices/{deviceId}/# and subscribe only to .../ack + .../cmd. Backend account can publish to all ack|cmd, subscribe to all evt|hb.
- **TLS on 8883** with a real cert (Caddy/Let's Encrypt). For the demo, run Mosquitto plaintext on 1883 *inside Docker network only* and document TLS as the production toggle — honest tradeoff for a student project.
- **Never put raw PII in payloads:** send uid + hashed/rotating BLE token, not student names. The backend maps uid/token -> studentId. RFID UUIDs and BLE tokens are identifiers, not secrets — treat them like usernames (see §19).

Example payloads:

```

// RFID tap uplink
{ "type":"rfid_tap", "eventId":"7f3a...", "deviceId":"ESP32-CR01-A3F2",
  "uid":"04:A3:1B:F2", "bootId":4821, "ts":"2026-09-05T09:01:12Z" }

// BLE batch uplink (15 s of passive scans, deduped per identity)
{ "type":"ble_batch", "eventId":"7f3b...", "deviceId":"ESP32-CR01-A3F2",
  "epoch":1938, "ts":"2026-09-05T09:40:00Z",
  "obs":[ { "token":"a91f...", "mac":"C4:7C:8D:11:22:33", "rssi":-64, "count":23},
    { "token":"b204...", "mac":"E4:...", "rssi":-81, "count":9} ] }

// Ack downlink

```

```
{ "type":"ack", "eventId":"7f3a...", "result":"OK_CHECKED_IN",
  "display":"WELCOME ANANYA", "beep":"success" }
// results: OK_CHECKED_IN | UNKNOWN_CARD | NO_ACTIVE_SESSION | DUPLICATE | LATE_CHECKIN | ERROR

// Heartbeat
{ "deviceId":"ESP32-CR01-A3F2", "uptimeS":12834, "freeHeap":182300,
  "wifiRssi":-58, "fw":"1.2.0", "ip":"10.0.4.21", "ts":"..." }
```

14. RFID Workflow

```
Tap -> RC522 reads UID (~50 ms) -> ESP32: card-present debounce (require same UID
2 reads 300 ms apart) -> check cooldown (ignore same UID for 5 s) -> build event
with eventId -> write to NVS outbox -> publish QoS 1 -> wait PUBACK -> delete from
outbox -> wait for backend ack -> drive OLED/buzzer from ack.result
```

Edge cases (backend unless noted):

- **Unknown card** -> ack UNKNOWN_CARD, red LED + buzz; event logged with student_id NULL for audit.
- **Double tap** -> ESP32 5 s cooldown suppresses; backend event_id / unique(class_session, student, checked_in) dedups. Same result either way — idempotent.
- **Tap after session start** -> allowed; flag late if > 10 min after start; dwell requirement prorated (see §16).
- **Tap outside classroom** -> the device only exists in one room; a tap means "at the door." That is inherent to RFID — accepted residual risk (see §20).
- **Wi-Fi down** -> NVS outbox, retry with backoff (1s, 2s, 5s, ..., 60s); on reconnect, replay in order. Backend handles replays via eventId.
- **MQTT down** -> same outbox; client auto-reconnect; LWT flips dashboard red.
- **Backend down** -> Mosquitto persists (queue for backend consumer subscribed QoS 1 + cleanSession=false); if broker also unreachable, ESP32 outbox holds thousands of events in NVS.
- **Duplicate messages** -> unique event_id -> no-op. Guaranteed by MQTT QoS 1 "at least once" semantics + dedup.

15. BLE Workflow — and the Critical Analysis

How ESP32 BLE scanning works [RESEARCH]: the ESP32 listens on 3 advertising channels. Legacy advert packets are ≤ 31 B; devices advertise on an interval (e.g., 100–1,000 ms); the scanner reports each MAC + payload + RSSI. Passive scan (no scan requests) is lower power and cheaper on airtime. NimBLE-Arduino is the right stack. There is no hard "max devices" — a classroom of 40 advertisers at ~500 ms intervals is comfortably within an ESP32's capacity **if** scan window/interval are tuned for Wi-Fi coexistence (window $\leq \sim 50\%$ of interval; e.g., 50 ms window / 110 ms interval).

The killer problem — identity: a phone advertises a *randomized* MAC and, at best, a rotating OS-level payload. You cannot map "MAC C4:7C:..." to "student #17" reliably, and even if you could today, the MAC changes. The fix used in production beacon systems: **identity must live in the advertisement payload, not the MAC.** A dedicated beacon keyfob can broadcast a fixed 128-bit service UUID + a per-student token in manufacturer data (fits in 31 B). A phone app can do the same — *while it runs*.

Practical options compared:

Option	Reliability	Cost	Difficulty	Privacy	Battery	Spoof resistance	Scales?
A. Smartphone app (Android foreground svc)	Medium — dies on battery savers, needs persistent notification	Rs.0	High (app + iOS problem)	Good (rotating token)	Medium drain	Medium (token in payload)	Painful across 100s of mixed devices
B. Dedicated BLE beacon keyfob per student	High — advertises 24/7, months on coin cell	Rs.250–350/student one-time	Low (no app!)	Medium (fixed MAC)	None for student	Medium-High (bind token <-> student server-side)	Excellent
C. BLE beacon card (dual with RFID card)	High	~Rs.300	Low	Medium	None	Medium-High	Excellent
D. Wi-Fi presence (probe frames)	Low — modern phones randomize MAC + randomized probes	Rs.0	Medium	Poor (sniffs everyone)	N/A	Low	No
E. RFID + periodic BLE (PDF's design, beacon-based)	High	as B	Low-Med	Medium	—	Medium-High	Excellent
F. QR code fallback	High per-scan, but one scan =	Rs.0 (print)	Low	Good	None	Low (photo shareable)	Good as fallback

	no dwell proof						
G. Hybrid (RFID + beacon/phone BLE + QR fallback)	Highest achievable at this cost	Rs.250–350/student	Medium	Medium	—	Medium-High	Excellent

[REC] Recommendation — Option G, staged:

1. **MVP demo:** Option B/C. Order a handful of cheap nRF51822-based keyfob beacons (~Rs.250–350, exactly the price the PDF itself footnotes) for demo students. They advertise a fixed service UUID + 8-byte token; no app, no OS dependency, works identically for every student.
2. **Stretch (NICE TO HAVE):** Android-only companion app using a foreground service advertising a rotating HMAC token — demonstrates the "bring your own phone" vision from the PDF. iPhone students use a beacon keyfob instead. Never promise iOS phone-as-beacon: Apple's platform makes it impossible to do honestly.
3. **QR fallback** for dead beacons/phones: teacher shows session QR; student scans -> creates CHECKED_IN with reduced dwell weight — keeps everyone markable.

[PDF] vs [RESEARCH] conflict, stated plainly: the PDF assumes "student's BLE beacon/smartphone signal" is continuously available and uniquely identifiable. With dedicated beacons this is true; with smartphones it is not, due to background advertising restrictions and MAC randomization. The smallest modification preserving all purchased hardware is: **same ESP32, same firmware scan code; change only what the student carries (Rs.250 beacon instead of / alongside the phone)**. The backend does not care either way — it consumes the same ble_batch observations.

16. RSSI + Dwell-Time Algorithm

Why raw "RSSI >= -75" is wrong [RESEARCH]: RSSI varies with body blocking (a phone in a pocket vs. on a desk can differ 10+ dB), orientation, multipath, and interference. One-threshold binary accumulation produces both false PRESENTs (phone left on desk near door) and false ABSENTs (phone in backpack under chair). Espressif's own coexistence docs note BLE scan interruption by Wi-Fi — the *sensor itself* introduces gaps. Any serious design needs smoothing + hysteresis + observation epochs.

[REC] The algorithm — epochs + EMA + dual threshold + confidence:

Definitions (configurable per classroom, stored on class_sessions):

- EPOCH = 15 s — one dwell decision unit
- RSSI_ENTER = -70 dBm (calibrated; PDF band -70...-78 -> enter at the top)
- RSSI_EXIT = -78 dBm (exit at the bottom -> 8 dB hysteresis band, per the PDF's own calibration range)
- EMA alpha = 0.3 — smoothed RSSI per student
- CONFIRM_K = 3 epochs present to leave TEMP_ABSENT; DROP_K = 3 epochs absent to flip MONITORING->TEMP_ABSENT
- Grace: GRACE = 5 min — the PDF's "< 5 min breaks pause accumulation" made precise: absent streaks <= 5 min (20 epochs) never reset dwell_secs and do not change state; they just do not add time.

```
every EPOCH (backend job, per active attendance_session):
  obs = ble_observations for (session, student, epoch)      # from batched uplinks
  if obs non-empty: rssi_avg = mean(max-rssi per observation)
  sm = EMA update: sm = alpha*rssi_avg + (1-alpha)*sm      # if no obs, sm decays toward -100

  present = (sm >= RSSI_ENTER) if state is present-side
               else (sm >= RSSI_EXIT)                      # hysteresis

  streak management:
    present -> present_streak++, absent_streak=0
    absent  -> absent_streak++, present_streak=0

  if state==MONITORING and absent_streak==DROP_K: state=TEMP_ABSENT
  if state==TEMP_ABSENT and present_streak==CONFIRM_K: state=MONITORING

  if present (and not TEMP_ABSENT): dwell_secs += EPOCH
    if dwell_secs >= prorated_required: state=VERIFIED; verified_at=now

  if absent_streak*EPOCH > GRACE: (still no reset - dwell is cumulative)
  if now > session.ends_at: state=CLOSED -> result PRESENT iff VERIFIED else UNVERIFIED/ABSENT
```

Proration for late arrivals [REC]: required = 2400 * remaining_fraction, min 20 min, so a student arriving at minute 30 is not auto-failed (PDF silent -> [ASSUMPTION] resolved this way; make it a config flag).

Why this is honest engineering: accumulation is *monotonic* (never resets — matches PDF), state changes require *streaks* (kills single-packet flapping), and the hysteresis band is exactly the PDF's -70/-78 calibration range formalized.

Calibration procedure (classroom, 30 min, both members):

1. Mount ESP32 at final position. Beacon/phone at 5 desk locations + door + hallway + adjacent room.
2. Log 10 min of RSSI per location (the ble_observations table is perfect for this).
3. Pick RSSI_ENTER ~ (weakest in-room desk value - 3 dB); RSSI_EXIT ~ (strongest hallway value - 3 dB). If bands overlap, the node needs placement further from the door/hall wall, or accept some error and rely on the 40-min cumulative test (robust to borderline RSSI — a hallway stander accumulates at the *edge* of the band, not solidly inside it).

17. Attendance State Machine

Server-authoritative (ESP32 mirrors only tap/LED states):



State	Meaning	Entry	Exit	Timeout	DB action	ESP32 mirror
IDLE	No session / waiting	boot, session end	rfid_tap	—	—	"TAP CARD" on OLED
CHECKED_IN	Tap accepted	valid tap, session live	first BLE obs	—	insert attendance_session	green beep "CHECKED IN"
MONITORING	Accumulating	BLE observations flowing	DROP_K absent streak -> TEMP_ABSENT; dwell met -> VERIFIED	session end	dwell updates	—
TEMP_ABSENT	Paused, grace running	DROP_K absent streak	CONFIRM_K present -> MONITORING; grace exceeded -> stays (no reset)	grace timer	state flip	"AWAY" if re-tapped
VERIFIED	40 min reached	dwell >= required	session end -> CLOSED	—	verified_at	celebratory beep + "PRESENT"
UNVERIFIED	Session ended, insufficient	session end without verify	—	—	result row	—
CLOSED	Final	session end	—	—	result view materializes	OLED "SESSION ENDED"
OFFLINE_PENDING	Network down	Wi-Fi/MQTT loss	reconnect+replay	—	outbox replay (firmware)	yellow LED

18. Offline / Failure Handling

Failure	Detection	Behavior
ESP32 loses Wi-Fi	WiFiEvent disconnect	NVS outbox; BLE scanning continues (radio freed for BLE — coexistence advantage); reconnect w/ backoff; replay
MQTT broker down	Publish fails / PUBACK timeout	Same outbox; ESP32 keeps sensing
Backend down (broker up)	Consumer absent; broker queues QoS 1 for backend's durable subscription	Events safe; dashboards show last data + device online
DB down	ORM errors	Consumer parks messages in dead_letter table; alert log; replay tool
Power loss / reboot	bootld increments	dwelL lives in DB -> zero loss; firmware resyncs clock via NTP/MQTT cmd; outbox survives in NVS
Student taps mid-outage	Tap event queued; immediate local feedback "SAVED — WILL SYNC"; no false negative	

ESP32 storage [REC]: use **Preferences (NVS)** for config (deviceld, creds, thresholds) and a small ring buffer of pending events (key-value, JSON-lines). LittleFS is fine if events grow large, but NVS is simpler and wear-leveled — avoid the deprecated EEPROM.h emulation. SPIFFS is deprecated; do not use it. No SD card needed.

Replay protocol: on reconnect, ESP32 publishes outbox events in order with original ts_device; backend uses eventId dedup and trusts ts_device for dwell timing (server clock only for ordering/latency metrics). Sequence: publish -> PUBACK -> **wait for**

backend ack (via ack topic) -> delete from NVS.

19. Security Architecture

Layer	Measures
MQTT	Per-device credentials; broker ACL (device can only touch its own topics); TLS 8883 (demo: internal plaintext + documented toggle); LWT
Firmware secrets	Device password compiled per-device via secrets.h from provisioning script (Member 1 flashes each board). Never commit secrets ; rotation script noted in README
API	JWT (15 min access) + refresh cookie (httpOnly, SameSite=Lax); bcrypt(12) password hashes; RBAC middleware per route
Injection/XSS/CSRF	Prisma parameterized queries (SQLi-safe by construction); React escapes output; CSRF token on cookie-auth refresh endpoint; rate-limit /auth/login (5/min/IP)
Replay/dupe events	eventId idempotency + server-side monotonic epoch checks + reject ts_device older than 24 h
RFID cloning	Honest posture: UID-only cards are cloneable, period. Mitigate cheaply: (a) write a signed secret block to each MIFARE card (HMAC-SHA256(uid, institution_key) into a data block) — cloned cards carry the *same* secret so pure cloning still works, but a *crafted* card fails; raises attack from "trivial" to "needs tooling"; (b) anti-proxy is fundamentally delivered by Layer 2 (dwell), not Layer 1 — say this in the demo
BLE spoofing	Token in adv payload is a per-student random 128-bit value stored hashed; rotate on re-issue; a sniffed token can be replayed by a determined attacker -> accept as residual risk, note that rotating tokens (phone app path) fix it
Fake ESP32	Per-device creds + ACL = cannot inject another room's events; backend additionally validates device<->classroom mapping
Audit	Every admin/teacher mutation -> audit_logs (before/after JSONB) — also the hackathon "responsible engineering" talking point
Privacy	Store token *hashes*, not raw BLE tokens; raw event logs admin-only; publish a short privacy note (tracking presence, not location)

20. Anti-Spoofing Strategy (honest assessment)

Attack	Difficulty	Mitigation	Fully preventable?
1. Give card to friend who taps for you	Trivial	None at Layer 1 — this is exactly what Layer 2 defeats : the real student's dwell only accumulates if their beacon/phone is present	Yes (if beacon is with the student)
2. Leave phone in class	Easy if phone used as beacon	Dedicated beacon is pocket-sized; static-signal heuristic flags it	No (nothing stops leaving a beacon) — see #3
3. Leave beacon in class, leave	Easy	Random teacher spot-checks + requirement that beacon RSSI pattern varies (static RSSI for 40 min = flag STATIC_SIGNAL for review) — cheap heuristic, high demo value	Mostly (residual: determined cheater)
4. Clone RFID UID	Easy (Flipper Zero)	Signed card block (§19) raises bar; Layer 2 still requires presence	Yes for *crafted* cards; no for *copied* cards (copy carries valid secret) — but proxy defeat still holds
5. Spoof BLE advertisements	Medium (needs sniffed token + ESP32/nRF tooling)	Token hashing + rotation on re-issue	No — accept and document
6. Stand by the door	Trivial	Door mounting + enter/exit thresholds calibrated to *desk* RSSI; hallway standers sit at hysteresis edge	Partially
7. Stand in hallway	Easy	Calibration (§16) + 40-min cumulative (hallway dwell is weak/noisy)	Mostly
8. Turn Bluetooth off	Trivial	Result: dwell stops -> UNVERIFIED. The system working as designed	N/A (self-defeating)
9. Change device	Easy	Token<->student binding; new device must be registered by admin	Yes

10. Fake ESP32 device	Hard (needs creds)	Per-device ACL + device registry	Yes
-----------------------	--------------------	----------------------------------	-----

Bottom line for the demo narrative: the system converts proxy attendance from "trivially easy" to "requires physical presence of a registered token for ~40 minutes, plus surviving a static-signal heuristic." That is a genuine, honest improvement — claim exactly that.

21. API Specification (REST — for humans; devices use MQTT)

Auth: Authorization: Bearer <jwt>; roles in parentheses.

Method	Endpoint	Auth	Body -> Response	Notes	
POST	/api/auth/login	public	{email,password} -> {accessToken} + refresh cookie	rate-limited	
POST	/api/auth/refresh	cookie	-> new access token	rotation	
GET/POST/PATCH	/api/students	ADMIN	CRUD; POST {user fields, rollNo, uid?, token?} -> student + optional card/identity creation	transactional	
GET/POST/PATCH	/api/teachers, /api/classrooms, /api/courses	ADMIN	standard CRUD	—	
GET/POST	/api/devices	ADMIN	POST {deviceId, classroomId} -> provisioning payload (mqtt creds)	Member 1 uses this to flash boards	
GET	/api/devices/:id	ADMIN,TEACHER	-> device + heartbeats + status	live from retained MQTT + DB	
POST	/api/sessions	TEACHER	{courseId, classroomId, startsAt, endsAt, thresholds?} -> session	also auto-startable from schedule	
POST	/api/sessions/:id/start /end	TEACHER	-> session status LIVE/CLOSED	publishes cmd to room device	
GET	/api/sessions/:id/live	TEACHER,ADMIN	-> {students: [{name,state,dwellSec s,smoothedRssi}], deviceStatus}	dashboard poll/WS seed	
GET	/api/sessions/:id/observations?studentId=	TEACHER	-> RSSI series	calibration view	
POST	/api/sessions/:id/corrections	TEACHER	{studentId, action, reason} -> audit-logged correction	never hard-delete	
GET	/api/attendance/me	STUDENT	-> my sessions, dwell, results	students see only self	
GET	/api/attendance/export?sessionId=	TEACHER	-> CSV	client-side print to PDF	
GET	/api/events?type=rfid\	ble	ADMIN	-> raw events, paginated	debugging
GET	/api/audit	ADMIN	-> audit logs	—	
WS	/socket.io	JWT	rooms: session:{id}, device:{id}, admin	engine emits state changes	

Errors: { "error": { "code": "UNKNOWN_CARD" | "VALIDATION" | "FORBIDDEN" | "NOT_FOUND" | "RATE_LIMITED", "message": "..." } } with proper HTTP codes.

22. Frontend Architecture

frontend/	
+-- src/	
+-- api/	# fetch wrapper, typed endpoints (generated from shared zod schemas)
+-- auth/	# AuthContext, useAuth, ProtectedRoute(role), login page
+-- ws/	# socket.ts – single Socket.IO client, room join/leave hooks
+-- pages/	
+-- admin/	# Students, Teachers, Classrooms, Devices (fleet map+health),
+--	# Card/Beacon mapping, AuditLogs, RawEvents explorer
+-- teacher/	# SessionsList, LiveSession (§23), History, Export
+-- student/	# MyAttendance (cards + per-subject %, history, current status)

```
| +-- components/      # StatusBadge, DwellRing, RssiSparkline, DataTable, ConfirmDialog
| +-- charts/          # Recharts wrappers (attendance %, trends)
```

State: TanStack Query for server data (caching + refetch), Socket.IO only for *live session* deltas merged into query cache. Loading skeletons everywhere; error toasts via a small event bus; forms validated with the same Zod schemas as the backend (single source of truth).

23. Real-Time Dashboard Architecture

```
ESP32 -> MQTT evt -> consumer -> AttendanceEngine -> UPDATE attendance_sessions
                                     +-> Socket.IO emit "session_delta" (room: session:{id})
                                     +-> DB write (source of truth)
React LiveSession page (joined room session:{id}) -> applies delta -> re-renders cards
Device status: retained MQTT status + heartbeat staleness job -> emit "device_delta" (room: admin)
```

Choice: Socket.IO over raw WS — automatic reconnect, room semantics, fallback transport; **over SSE** because corrections/commands flow client->server later; **over polling** because 15 s refresh feels dead at a demo table. Polling remains as the seeded initial fetch (GET /sessions/:id/live powers first paint; WS keeps it alive).

Live session UI: one card per enrolled student — name, avatar-initial, state badge (color from FSM state), live dwell ring (dwell/2400), smoothed-RSSI sparkline, "last seen" timestamp; top strip: device health pill (green/yellow/red), session countdown, verified count.

24. ESP32 Device Management

- Registry table (\$12) + heartbeat every 30 s: {uptimeS, freeHeap, wifiRssi, ip, fw} -> QoS 0 to hb.
- Status derivation: ONLINE = heartbeat < 90 s; DEGRADED = online but wifiRssi < -75 or freeHeap below threshold or outbox backlog reported; OFFLINE = LWT "offline" or heartbeat stale.
- Firmware version reported -> admin UI shows fleet table with fw drift highlighted.
- Provisioning flow (Member 1): generate per-device creds in admin UI -> flash script writes secrets.h + deviceId -> device appears on first heartbeat.
- OTA: **NICE TO HAVE** (ArduinoOTA or HTTP OTA endpoint) — note in roadmap, do not build for MVP.

25. Hardware Simulator

Build this on day 2-3. It unblocks ~80% of your work.

simulator/ — a Node.js CLI (same mqtt.js + shared Zod schemas as backend; same topic/payload contract as firmware):

```
sim.js --device ESP32-CR01-A3F2 --scenario attendance_demo_42min
```

Scenarios as plain JS functions with sleep():

- tap(studentUid) — publish rfid_tap with eventId
- bleBatch([token, mac, rssi]) — publish 15 s epoch batches; RSSI values from realistic profiles (desk -62±4, pocket -70±6, hallway -80±5, static -63±0.5)
- walkOut(), walkIn(), shortBreak(3min), longAbsence(12min), phoneDies()
- wifiOutage(min) — stop publishing (backend must tolerate), then replay storm with old eventIds (tests dedup)
- duplicateFlood() — replay same event 5x (tests idempotency)
- reboot() — bootId++ mid-session (tests continuity)

Why it works forever: the simulator and the real ESP32 are indistinguishable to the backend — same topics, same payloads, same idempotency contract. Integration tests run against the simulator; hardware integration later is a *contract test*, not a rewrite.

26. Testing Strategy

Firmware/HIL (both members): RC522 read reliability (100 taps, 0 misreads); debounce; outbox replay after pulled Wi-Fi; reboot mid-session.

MQTT contract: unknown device creds rejected (ACL); wrong-topic publish denied; QoS 1 replay dedup; LWT flips on kill -9 of simulator.

Backend unit (Vitest): dwell algorithm table-tests — the money tests:

#	Scenario	Expected
T1	tap + 42 solid min in-room	CHECKED_IN->MONITORING->VERIFIED->CLOSED=PRESENT
T2	tap + immediate exit, never returns	dwell~0 -> UNVERIFIED

T3	tap + 3-min break at minute 20	dwel continues after; verified at ~43 min elapsed
T4	tap + 12-min absence	grace exceeded -> TEMP_ABSENT; return needs CONFIRM_K; still cumulative
T5	30 consecutive borderline epochs (-76 dBm)	hysteresis keeps state, no flapping
T6	duplicate rfid_tap x5	single attendance_session
T7	replay of ble_batch after 10-min outage (same eventId)	no double dwell
T8	unknown card	event logged, no session, ack UNKNOWN_CARD
T9	tap with no active session	ack NO_ACTIVE_SESSION
T10	hallway RSSI profile (-80±5) for 60 min	dwel stays ~0 (calibration-dependent)
T11	static RSSI beacon all session	PRESENT + STATIC_SIGNAL review flag
T12	session ends mid-monitoring	CLOSED, result per state

API (Supertest): auth flows, RBAC denials (student hitting admin route), validation, corrections audit trail.

E2E (Playwright, optional): teacher login -> create session -> simulator runs T1 -> dashboard shows verified.

Load: simulator with 60 students x 4 epochs/min = 240 msg/min/room — trivial; verify no broker/DB strain at 20 rooms.

Security: npm audit, ZAP quick scan on API, ACL negative tests.

27. Development Roadmap (one software dev; parallel tracks with hardware)

Phase	Objectives	Key deliverables	Depends on	Effort	Parallel w/ hardware?
0 Architecture	freeze MQTT schema + repo skeleton	docs/mqtt-contract.md, monorepo	—	2 d	yes — agree contract with Member 1
1 Simulator	sim.js with T1-T12 scenarios	simulator/	Phase 0	2-3 d	yes
2 Database	Prisma schema + migrations	schema.prisma, seed	Phase 0	2 d	yes
3 MQTT infra	Mosquitto Docker, ACLs, TLS toggle	docker-compose, acl file	Phase 0	1-2 d	yes
4 Backend core	consumer, auth, RBAC, devices CRUD	working API skeleton	2,3	4-5 d	yes
5 Attendance engine	FSM + dwell algorithm + unit tests T1-T12	engine + green tests	1,2,4	4-6 d	no hardware needed
6 REST APIs	sessions, live, corrections, export	full API	5	3 d	yes
7 Frontend	admin/teacher/student UIs	SPA	6	5-7 d	yes
8 Realtime	Socket.IO live dashboard	live view	6,7	2-3 d	yes
9 Security pass	rate limits, audit, secrets hygiene	hardening	4-8	2 d	yes
10 Firmware integration	real ESP32 on broker; contract test	HIL tests pass	Member 1 firmware	3-5 d	joint
11 Calibration	classroom RSSI walk-test -> thresholds	calibrated config	10	1 d	joint
12 Full E2E + pilot	real class dry-run, fix bugs	stable demo	11	3-4 d	joint
13 Deployment + demo prep	VPS, CI, demo script + fallback video	live system	12	2-3 d	—

What to code FIRST (exact order): (1) MQTT contract doc -> (2) docker-compose with Mosquitto+Postgres -> (3) Prisma schema -> (4) simulator -> (5) consumer + raw event tables -> (6) engine with unit tests -> (7) REST -> (8) frontend -> (9) realtime -> (10) firmware hookup. You can reach step (9) without touching hardware.

28. GitHub Repository Structure

```
smart-attendance/
+-- README.md           # architecture diagram, quickstart, demo script
+-- docs/
```

```

| +-- mqtt-contract.md # THE contract (topics, payloads, QoS, idempotency)
| +-- state-machine.md # FSM + dwell algorithm spec
| +-- calibration.md # RSSI procedure + results
| +-- api.md # endpoint reference
+-- backend/ # $11 structure
+-- frontend/ # $22 structure
+-- simulator/ # $25
+-- firmware/ # Member 1's PlatformIO/Arduino project (NimBLE, NVS outbox)
+-- deploy/
| +-- docker-compose.yml # mosquitto, api, postgres, caddy
| +-- mosquitto/ (acl, passwd gen script)
| +-- github-actions/ # SSH deploy workflow
+-- shared/ # zod schemas + TS types shared backend/frontend/simulator
+-- .env.example

```

Env vars: DATABASE_URL, JWT_SECRET, MQTT_URL, MQTT_BACKEND_USER/PASS, CORS_ORIGIN. Secrets only via env/Docker secrets; firmware secrets via untracked secrets.h.

29. Required Development Tools (minimum)

Tool	Purpose	Needed when
Node.js 20 + pnpm	backend, frontend, simulator	day 1
Git + GitHub	everything	day 1
VS Code (+ Prisma, ESLint extensions)	editor	day 1
Docker Desktop	Mosquitto + Postgres local	day 1
MQTT Explorer (GUI) or mqttx CLI	watching topics during dev	day 2
Postman/Insomnia or curl/httpyac	REST testing	Phase 4
PostgreSQL client (pgAdmin/psql, or Prisma Studio)	DB inspection	Phase 2
PlatformIO (VS Code ext) or Arduino IDE	firmware (Member 1)	Phase 10 (install anytime)
nRF Connect mobile app	verifying beacons/advertising	calibration

Skip: ESP-IDF (NimBLE-Arduino is enough), Kubernetes, Terraform, Prometheus (until it hurts).

30. Deployment Strategy

- **Dev:** laptop, Docker Compose (mosquitto+postgres+api+frontend via Caddy).
- **Prod/demo:** single VPS (Hetzner CX22 ~EUR 4/mo or DO \$6). Compose: Caddy (TLS + reverse proxy, auto-cert), API, Postgres (volume, nightly pg_dump cron + offsite), Mosquitto (8883 TLS + 1883 internal). GitHub Action on main: build images, SSH, docker compose pull && up -d, run Prisma migrate.
- **Fallback for zero-ops:** Railway/Render template deploy — acceptable for the demo; VPS preferred for the "we control the broker" security story.
- Backups: nightly pg_dump to object storage; before-demo snapshot. This is the disaster recovery for the hackathon.

31. Scalability Analysis

Per room traffic: evt batches 4/min (QoS 1) + hb 2/min + acks = **< 20 msg/min/room**, few KB/min. Utterly trivial for Mosquitto (handles 100k+ msg/s).

Classrooms	ESP32 nodes	MQTT msgs/min	DB writes/day	DB size/semester	Infra needed
1	1	~20	~50k events	< 500 MB	laptop-grade
5	5	100	250k	~2 GB	\$6 VPS
20	20	400	1 M	~8 GB	same VPS; index tuning
100	100	2k	5 M	~40 GB	\$12-24 VPS, Postgres tuning
500	500	10k	25 M	~200 GB	dedicated/managed DB; broker cluster only beyond ~5k rooms

The architecture scales vertically far beyond your scope; the horizontal story (multiple API instances, managed MQTT) exists but is explicitly out of scope. The ESP32 scan load (40 advertisers/room) does not multiplex across rooms — each node is independent. The real bottleneck at scale is **fleet ops** (provisioning, OTA, calibration) — architect the device registry for it now, solve it later.

32. MVP Scope

MUST HAVE

- ESP32 node: RC522 tap -> OLED/buzzer feedback <- MQTT ack; NimBLE passive scan -> batched ble_batch; NVS outbox + replay; heartbeat + LWT
- Backend: consumer, engine (FSM + dwell), auth/RBAC (3 roles), sessions CRUD, live + history APIs
- Web: teacher live dashboard + session history/export; student my-attendance; admin student/card/beacon/device management
- Simulator with T1-T12 scenarios
- Calibration done in the real classroom; thresholds in DB
- Dockerized deployment on one VPS; TLS toggle; audit logs

SHOULD HAVE

- Dedicated beacon keyfobs for all demo students (Option B) — *technically MVP for honesty, staged by budget*
- QR fallback for missing beacons
- STATIC_SIGNAL review heuristic
- Signed MIFARE card block
- CI deploy pipeline

NICE TO HAVE

- Android foreground-service app with rotating token
- OTA updates, device firmware drift UI
- Playwright E2E suite, per-student RSSI analytics page

DO NOT BUILD YET

- iOS app (platform makes it futile for background adv)
- Multi-tenant SaaS, mobile dashboard apps, ML-based localization, Kubernetes, per-seat localization, camera verification, LoRa/4G backhaul, microservices, QoS 2, Sparkplug

33. Future Improvements

Rotating BLE tokens (TOTP-style) to kill sniff-and-replay; per-teacher "spot check" mode (random 30-s RSSI burst requiring response); Wi-Fi RTT / Channel Sounding for true ranging; ultrasonic co-presence token (phone + ESP32 speaker); fusion with door counters; per-room self-calibration using known beacon placements; edge model flagging anomalous RSSI for review; integration with academic calendars for auto-scheduled sessions; parent/student dispute workflow with observation evidence (the raw ble_observations tables are exactly the evidence store for this).

34. Major Technical Risks

1. **Beacon availability/cost** — phones cannot be trusted; budget for keyfobs (the PDF already priced them).
2. **RSSI calibration variance across rooms** — mitigation: per-room thresholds + the cumulative 40-min test is forgiving; do not over-promise per-seat accuracy.
3. **ESP32 coexistence quirks** (scan gaps during Wi-Fi bursts) — mitigation: passive scan, 50% duty cycle, batched uplinks; validate with the 20-40 advertiser test early (use 3 spare ESP32s or phones with nRF Connect as advertisers).
4. **OEM battery killers killing the phone-app path** — mitigation: beacon-first strategy; app is optional garnish.
5. **Wi-Fi infrastructure** in classrooms (captive portals, WPA2-Enterprise) — **[ASSUMPTION]** lab/hotspot network; verify EAP support early; fallback: phone hotspot demo.
6. **Demo-day Murphy** — mitigation: recorded fallback video + local hotspot isolated network + simulator can drive the whole dashboard live on stage.

35. Final Recommendations — and the answer to Question 33

Can RFID + ESP32 BLE RSSI + 40-min dwell give reliable anti-proxy attendance?

As literally specified (student smartphones as the BLE source): NO. Three independent platform-level facts make it impossible to honestly promise: (1) iOS forbids meaningful background BLE advertising entirely — no custom payloads, overflow-area only, receiver screen-on required. (2) Android allows it only via a persistent-notification foreground service, and OEM battery managers (Samsung foremost) kill it within minutes, sometimes silently after onStartSuccess. (3) MAC randomization means identity cannot live in the MAC anyway — it must live in the payload, which requires an app, which loops back to (1) and (2).

With the smallest possible modification: YES, under stated conditions. The smallest change that preserves 100% of the purchased hardware: **students carry a Rs.250-350 BLE beacon keyfob** (the exact option the PDF footnotes) that broadcasts a fixed service UUID + per-student token. Then:

- Reliable: dedicated beacons advertise 24/7 for months on a coin cell, immune to OS background policies.
- The system works precisely when the *combination* holds: RFID tap (intent, at the door) + beacon present with smoothed RSSI

inside calibrated thresholds for a cumulative 40 of ~50–60 min (presence).

- Residual cheats (leave beacon on desk) are detectable via static-signal heuristics and spot checks, and are far costlier than today's one-second proxy tap.
- It proves **room-level sustained presence**, not seat-level occupation, and not identity-with-certainty (tokens can theoretically be sniffed/replayed). Claim exactly this on demo day — it is still a large, honest improvement over RFID-only systems.

Top 5 action items this week:

1. Agree the MQTT contract doc (§13) with Member 1 — it is your entire interface.
2. Stand up Docker (Mosquitto+Postgres) and scaffold the repo (§28).
3. Build the simulator (§25) — unblocks everything.
4. Write the attendance engine with the T1–T12 unit tests before any UI exists.
5. Order 3–5 nRF51822-class beacon keyfobs now so they arrive before integration week — and test one against the ESP32's NimBLE scan with 40 advertisers present.

End of blueprint. This document is deliberately conservative: one VPS, one broker, one Postgres, one SPA, one firmware — every component justified by "why not simpler", and every risky assumption replaced with research, a calibration procedure, or an honest limitation statement.